

Entrega # 2

Alumnos:

Juan Sebastián Villa Rodas.

(1027804015)

Miguel Ángel Guarnizo Salazar.

(1038868741)

Tomas Buritica Jaramillo.

(1033179934)

Docente:

Juan Esteban Gómez Tirado.

Materia:

ANÁLISIS Y DISEÑO DE ALGORITMOS.

Año:

2026-1

Tabla Contenidos

1. Portada	Pg1.
2. Introducción	Pg3.
3. Definición del problema	Pg3.
4. Entradas, salidas y restricciones	Pg4.
5. Casos de prueba	Pg5.
6. Solución por Greedy	Pg6.
7. Complejidad temporal y espacial	Pg7.
8. Solución Backtracking.....	Pg8.
9. Poda de ramas	Pg8.
10. Complejidad recursiva	Pg9.
11. Medición empírica	Pg10.
12. Comparación greedy vs backtracking.....	Pg11.
13. Explicación de la gráfica greedy vs backtracking.....	Pg12.
14. Conclusiones	Pg13.

INTRODUCCIÓN

En esta segunda entrega se implementaron dos nuevos paradigmas de programación:

- Greedy
- Backtracking

El objetivo fue comparar diferentes formas de resolver el problema de planificación de horarios y analizar ventajas, desventajas y rendimiento de cada enfoque.

DEFINICIÓN DEL PROBLEMA

El problema consiste en generar horarios universitarios válidos para un conjunto de materias.

Cada materia debe ser asignada a:

- un salón,
- un horario,
- un profesor,
- un grupo.

Todo esto respetando ciertas restricciones para evitar conflictos.

El sistema debe encontrar asignaciones válidas sin superponer horarios entre profesores, grupos o salones.

ENTRADAS

Las entradas utilizadas por el programa son:

- Lista de materias.
- Cantidad de alumnos por materia.
- Profesores disponibles.
- Grupos académicos.
- Salones disponibles.
- Capacidad de los salones.
- Horarios disponibles.

SALIDAS

La salida consiste en un horario válido para cada materia, mostrando:

- nombre de la materia,
- profesor asignado,
- grupo,
- salón,
- horario.

RESTRICCIONES

El sistema debe cumplir las siguientes restricciones:

- Un profesor no puede dictar dos materias al mismo tiempo.
- Un grupo no puede recibir dos clases simultáneamente.
- Un salón no puede ser utilizado por dos materias en el mismo horario.
- El salón debe tener capacidad suficiente para la cantidad de estudiantes.

CASOS DE PRUEBA

Se utilizaron tres conjuntos de datos para probar el comportamiento de los algoritmos:

Caso	Materias	Salones	Profesores	Grupos	Horarios
Pequeño	3	2	2	2	6
Mediano	8	4	5	4	14
Grande	15	6	8	6	22

Estos casos permitieron analizar cómo cambia el tiempo de ejecución cuando aumenta el tamaño de entrada.

SOLUCIÓN GREEDY

El algoritmo greedy toma decisiones locales óptimas.

Para cada materia:

- Selecciona el primer salón disponible.
- Asigna el primer horario válido.
- Y continúa con la siguiente materia.

La idea principal es encontrar soluciones rápidamente sin explorar demasiadas combinaciones,

DECISIÓN LOCAL ÓPTIMA

La decisión local óptima consiste en:

- Asignar inmediatamente el primer horario válido encontrado.

Esto reduce el tiempo de búsqueda y mejora la velocidad del algoritmo.

```
def greedy(materias, salones, horarios):
    asignaciones = []

    # Ordenar materias por cantidad de alumnos (descendente)
    materias_ordenadas = sorted(
        materias,
        key=lambda x: x["alumnos"],
        reverse=True
    )

    for materia in materias_ordenadas:
        for salon in salones:
            if salon["capacidad"] >= materia["alumnos"]:
                for horario in horarios:
                    conflicto = hay_conflicto(
                        asignaciones,
                        materia["profesor"],
                        materia["grupo"],
                        salon["nombre"],
                        horario
                    )
                    if not conflicto:
                        asignaciones.append({
                            "materia": materia["nombre"],
                            "profesor": materia["profesor"],
                            "grupo": materia["grupo"],
                            "salon": salon["nombre"],
                            "horario": horario
                        })
                        break
                if any(a["materia"] == materia["nombre"] for a in asignaciones):
                    break

    return asignaciones
```

https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Disc-o/blob/main/2-src/greedy.py

COMPLEJIDAD TEMPORAL Y ESPACIAL

Complejidad Temporal

El algoritmo greedy realiza las siguientes operaciones:

- **Ordenamiento:** La operación sorted tiene complejidad $O(n \log n)$, donde n es el número de materias.
- **Bucles anidados:** Al igual que la fuerza bruta, se tienen tres bucles anidados (materias, salones, horarios) con verificación de conflictos $O(n)$

$$T_{\text{greedy}}(n, s, h) = O(n \log n) + O(n^2 \cdot s \cdot h) = O(n^2 \cdot s \cdot h) = O(n^2):$$

Complejidad Espacial

La complejidad espacial está determinada por:

- La lista de asignaciones: almacena máximo n elementos, por lo tanto $O(n)$.
- Las estructuras de entrada (materias, salones, horarios): $O(n + s + h)$.
- No se utilizan estructuras auxiliares adicionales significativas.

$$S(n) = O(n + s + h) = O(n)$$

SOLUCIÓN BACKTRACKING

El algoritmo backtracking explora diferentes combinaciones posibles.

Cuando encuentra un conflicto:

- retrocede
- prueba otra opción
- y continúa buscando soluciones válidas.

PODA DE RAMAS

La poda ocurre cuando:

- Un profesor ya está ocupado
- Un grupo tiene conflicto
- Un salón ya fue asignado.

En esos casos, el algoritmo deja de explorar esa rama para ahorrar tiempo.

```
def backtracking(materias, salones, horarios, indice, asignaciones):
    # Caso base: todas las materias asignadas
    if indice >= len(materias):
        return True

    materia = materias[indice]

    for salon in salones:
        # Poda: verificar capacidad
        if salon["capacidad"] >= materia["alumnos"]:
            for horario in horarios:
                conflicto = hay_conflicto(
                    asignaciones,
                    materia["profesor"],
                    materia["grupo"],
                    salon["nombre"],
                    horario
                )

                if not conflicto:
                    # Asignar
                    nueva = {
                        "materia": materia["nombre"],
                        "profesor": materia["profesor"],
                        "grupo": materia["grupo"],
                        "salon": salon["nombre"],
                        "horario": horario
                    }
                    asignaciones.append(nueva)

                    # Recursion: intentar asignar la siguiente materia
                    resultado = backtracking(
                        materias, salones, horarios,
                        indice + 1, asignaciones
                    )

                    if resultado:
                        return True # Solucion encontrada

                    # Backtrack: deshacer asignacion
                    asignaciones.pop()

    return False # No se encontro solucion por este camino
```


https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-o/blob/main/2-src/backtracking.py

COMPLEJIDAD TEMPORAL BACKTRACKING

En el peor caso, el backtracking explora todo el espacio de soluciones sin poda efectiva, lo que equivale a generar todas las permutaciones posibles de asignaciones:

$$T_{backtracking}^{peor}(n, s, h) = O((s \cdot h)^n) \triangleup \triangleleft$$

Caso promedio con poda: Con las restricciones de poda implementadas, el algoritmo suele comportarse mucho mejor que el peor caso teórico. Las restricciones de conflicto reducen significativamente el espacio de búsqueda en instancias realistas:

$$T_{backtracking}^{promedio}(n, s, h) \ll O((s \cdot h)^n) \triangleup \triangleleft$$

COMPLEJIDAD ESPACIAL

La complejidad espacial está determinada por:

- La lista de asignaciones: almacena máximo n elementos, por lo tanto $O(n)$.
- Las estructuras de entrada (materias, salones, horarios): $O(n + s + h)$.
- La pila recursiva: puede alcanzar una profundidad máxima de n llamadas recursivas.

$$S(n) = O(n + s + h) = O(n)$$

MEDICIÓN EMPÍRICA

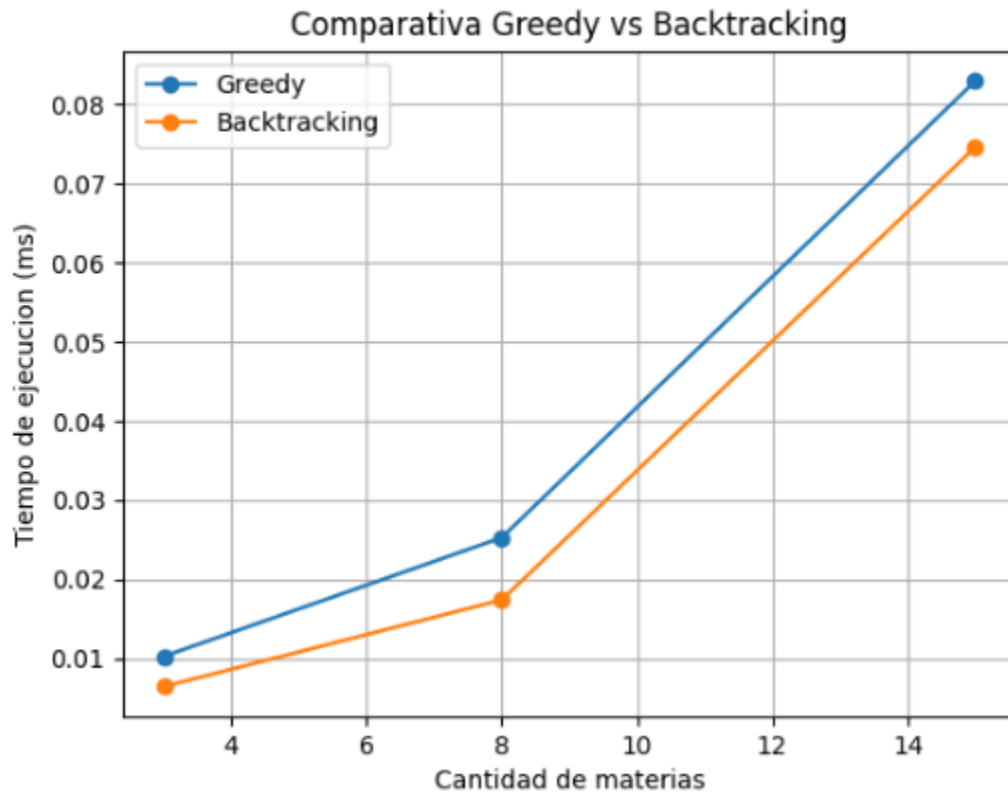
Para realizar la medición empírica se ejecutaron ambos algoritmos utilizando los mismos casos de prueba empleados anteriormente. Se registró el tiempo de ejecución en milisegundos utilizando la biblioteca time de Python.

Caso	Materias	Greedy (ms)	Backtracking (ms)
Pequeño	3	0.0119	0.0074
Mediano	8	0.0236	0.0167
Grande	15	0.0818	0.0730

COMPARATIVA GREEDY VS BACKTRACKING

Característica	Greedy	Backtracking
Tipo de implementación	Iterativa	Recursiva
Facilidad de comprensión	Alta	Media
Organización del código	Básica	Más modular
Uso de memoria	Menor	Mayor
Tiempo de ejecución	Muy rápido	Más lento
Complejidad temporal	$O(n^2)$	$O(b^n)$
Complejidad espacial	$O(n)$	$O(n)$ + pila recursiva
Ventaja principal	Rapidez	Explora más soluciones
Desventaja principal	No garantiza solución óptima	Mayor costo computacional

GRAFICA FUERZA GREEDY VS BACKTRACKING



https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-o/blob/main/4-graficas/greedy_vs_backtracking.png

La gráfica muestra el crecimiento de los tiempos de ejecución de greedy y backtracking a medida que aumenta la cantidad de materias.

Se puede observar que:

- greedy mantiene tiempos muy bajos debido a sus decisiones rápidas,
- backtracking tarda un poco más porque explora más combinaciones,
- ambos algoritmos aumentan progresivamente su tiempo de ejecución en el caso grande.

CONCLUSIONES

En esta entrega se implementaron dos nuevos paradigmas de programación para el problema de planificación de horarios universitarios: greedy y backtracking.

El algoritmo greedy destacó principalmente por su rapidez y simplicidad, mientras que backtracking permitió explorar una mayor cantidad de soluciones posibles gracias al uso de poda y retroceso.

Las pruebas realizadas permitieron comparar el comportamiento de los algoritmos y analizar cómo cambia el tiempo de ejecución conforme aumenta el tamaño de entrada.

Finalmente, esta entrega ayudó a comprender mejor las diferencias entre algoritmos orientados a velocidad y algoritmos orientados a exploración de soluciones, permitiendo analizar las ventajas y limitaciones de cada enfoque dentro del mismo problema.